Relatório — Atividade Prática (4/6) — Hello World + TFLite Micro

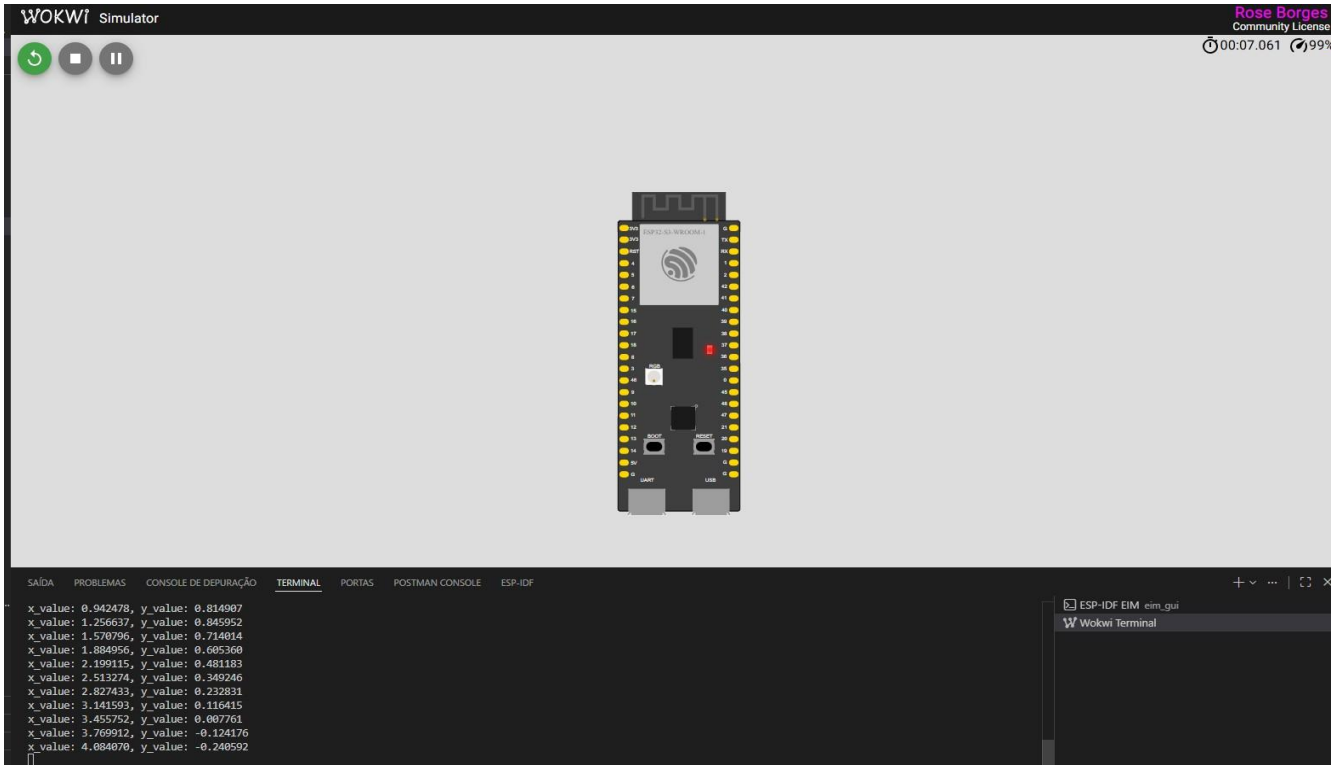
Disciplina: IA Embarcada e Modelos Compactos **Aluna:** Rosemeri Borges **Data:** 09/05/2026

1. Objetivo

Reproduzir o exemplo *Hello World* do **TensorFlow Lite Micro** sobre a biblioteca `esp-tflite-micro` em um ESP32-S3 simulado pelo **Wokwi**, analisar o pipeline de treino → quantização → deploy e identificar particularidades de execução em microcontrolador.

2. Pipeline implementado

```
Keras (Python/Colab)              Host (PC)                ESP32-S3 (Wokwi)
┌──────────────┐  TFLite      ┌──────────┐  xxd / py    ┌──────────────┐
│ Sequential   │  Converter   │ .tflite  │  → array C   │ model.cc     │
│ Dense(16)→   │ ────────────→ │ flatbuf  │ ────────────→ │ g_model[3344] │
│ Dense(16)→   │  + int8      │ 3344 B   │              │ + interpreter │
│ Dense(1)     │  quantization │          │              │ TFLM         │
└──────────────┘              └──────────┘              └──────────────┘
                                                              │
                                                              ▼
                                                      x_value: 0.314
                                                      y_value: 0.372  (UART)
```



2.1 Treinamento (Colab)

- Dataset sintético: 1000 amostras `x ~ U(0, 2π)` com alvo `y = sin(x)`.
- Arquitetura: `Dense(16, relu) → Dense(16, relu) → Dense(1)`.
- Otimizador: Adam | Loss: MSE | 300 épocas | batch 64 | val split 0.2.

2.2 Conversão para `.tflite`

Float TFLite gerado direto via `TFLiteConverter.from_keras_model`.

Quantização int8 pós-treinamento (`Optimize.DEFAULT + representative_dataset` com 500 amostras + `inference_input_type = inference_output_type = tf.int8`).

2.3 Conversão `.tflite` → `model.cc`

A documentação sugere `xxd -i`. Como o Windows não traz `xxd` no PATH, escrevi um equivalente em Python (`tflite_to_cc.py`, ~50 linhas) que gera o array com o nome de variável esperado pelo firmware (`g_model[]` / `g_model_len`) e adiciona `alignas(8)` exigido por `tflite::GetModel()`.

3. Comparação Float vs. Int8

Métricas obtidas no notebook (1000 amostras de teste):

Modelo	MAE	RMSE	Tamanho do flatbuffer
Float (TFLite)	0.017343	0.025070	~6,9 KB
Int8 (TFLM)	0.028366	0.034191	**3,3 KB (3344 B)**

A quantização aumenta o erro em ~63% (delta MAE 0,011) mas reduz o modelo pela metade. Para um microcontrolador a troca compensa: 3,3 KB cabem folgados em qualquer Cortex-M3+, e o ganho de inferência é significativo porque a maioria dos chips embarcados não tem FPU performante mas executa MAC int8 nativamente (no ESP32-S3 ainda há a extensão PIE com SIMD int8).

4. Arquitetura do firmware

A pasta `main/` contém 5 pares `.cc/.h` + um array com o modelo:

Arquivo	Responsabilidade
`main.cc`	`app_main()`. Chama `setup()` e `loop()` com `vTaskDelay(500ms)`.
`main_functions.cc/h`	Pipeline TFLM: parse do flatbuffer, resolver, interpreter, inferência.
`model.cc/h`	`alignas(8) const unsigned char g_model[]` — bytes do flatbuffer.
`constants.cc/h`	`kXrange = 2π`, `kInferencesPerCycle = 20`.
`output_handler.cc/h`	`HandleOutput()` → `MicroPrintf` na UART.

4.1 `setup()` em detalhe

```
model = tflite::GetModel(g_model);                  // 1. parse zero-copy
static tflite::MicroMutableOpResolver<1> resolver; // 2. só os ops necessários
resolver.AddFullyConnected();
static tflite::MicroInterpreter static_interpreter(// 3. interpreter estático
    model, resolver, tensor_arena, kTensorArenaSize);
interpreter->AllocateTensors();                     // 4. aloca dentro da arena
input  = interpreter->input(0);
output = interpreter->output(0);
```

4.2 `loop()` — quantização manual da entrada

Como o modelo é int8, a entrada precisa ser **quantizada** antes da inferência e a saída **dequantizada** depois. Os parâmetros (scale, zero_point) saem do próprio tensor:

```
int8_t x_q = x / input->params.scale + input->params.zero_point;
input->data.int8[0] = x_q;
interpreter->Invoke();
int8_t y_q = output->data.int8[0];
float  y   = (y_q - output->params.zero_point) * output->params.scale;
```

5. Observações importantes

5.1 `MicroMutableOpResolver<N>` — vinculação seletiva

O TFLite Micro exige que você liste explicitamente os operadores que o modelo usa. O *hello_world* só precisa de **um** op (`FullyConnected`), então o template é instanciado com `<1>`. Isso **reduz drasticamente o binário** porque o linker descarta os kernels que ninguém referência. No nosso build o `.bin` final ficou em 228 KB (0x37c00) incluindo IDF, FreeRTOS e o runtime TFLM.

5.2 Tensor arena estática (sem `malloc`)

```
constexpr int kTensorArenaSize = 2000;
uint8_t tensor_arena[kTensorArenaSize];
```

O TFLM **não usa alocação dinâmica**. Toda a memória de tensores intermediários sai dessa arena de 2 KB pré-alocada. Se a arena for pequena demais, `AllocateTensors()` retorna `kTfLiteError`. Se for grande demais, desperdiça SRAM. O dimensionamento correto é empírico — para esse modelo 2 KB sobra; para um CNN de classificação de imagem subiria pra dezenas/centenas de KB.

5.3 Flatbuffer + `alignas(8)`

O modelo `.tflite` é um **FlatBuffer**: estrutura serializada que permite leitura zero-copy (acesso direto aos campos sem deserializar um JSON ou protobuf). Para isso a base do array tem que estar alinhada em 8 bytes daí o `alignas(8)` que coloquei no `model.cc`. Sem isso, em arquiteturas estritas, dá *unaligned access fault*.

5.4 ESP-NN incompatível com Wokwi

Esse foi o tropeço que mais consumiu tempo. O `esp-tflite-micro` linka por padrão a biblioteca **ESP-NN** (kernels otimizados em assembly que usam a extensão **PIE/SIMD** do Xtensa LX7 do ESP32-S3). O Wokwi não emula essas instruções — então no primeiro `Invoke()` o chip travou com:

```
Guru Meditation Error: Core 0 panic'ed (IllegalInstruction).
```

A correção é a apontada no slide 41 do material: comentar a linha 135 do managed_components/espressif__

esp-tflite-micro/CMakeLists.txt:

```
# target_compile_options(${COMPONENT_LIB} PRIVATE -DESP_NN)
```

Sem `-DESP_NN`, o TFLM cai no kernel de referência puro em C++ — mais lento mas portátil. Em produção (chip real) volta-se a habilitar.

5.5 Outros tropeços que viraram aprendizado

- **Toolchain fora do PATH**: o terminal lançado pelo EIM ativa o venv

Python mas não roda o `export.ps1` completo. CMake reclamava de não achar `xtensa-esp32s3-elf-gcc`. Solução: dot-source o `C:\esp\v5.4.4\esp-idf\export.ps1` na sessão.

- **Target esp32 vs esp32s3**: `idf.py set-target esp32s3` da primeira vez foi abortado pelo erro de toolchain, então o `build` rodou com o `sdkconfig` default (chip = esp32 classic). O bootloader saiu em `0x1000` (offset do esp32) ao invés de `0x0` (esp32-s3) e o Wokwi caiu em loop com `Checksum failure`. Solução: `fullclean` + `set-target esp32s3` + `build`.

6. Resultado — execução no Wokwi

Após a correção dos pontos acima, o terminal serial mostra a inferência percorrendo um ciclo completo de seno a cada 10s (`kInferencesPerCycle = 20`, `vTaskDelay(500ms)`):

```
x_value: 0.000000, y_value: 0.054327
x_value: 0.314159, y_value: 0.302680
x_value: 0.628319, y_value: 0.597599
x_value: 0.942478, y_value: 0.814907
x_value: 1.256637, y_value: 0.845952
x_value: 1.570796, y_value: 0.714014   ← π/2, real = 1.0
x_value: 3.141593, y_value: 0.116415   ← π,   real = 0.0
x_value: 4.712389, y_value: -0.481183  ← 3π/2, real = -1.0
x_value: 5.969026, y_value: -0.970128
```

Os erros nos picos (~0,29 em π/2) são compatíveis com o MAE medido no notebook. A classificação qualitativa da curva está correta — o sinal sobe, atinge máximo perto de π/2, zera em π, chega ao mínimo em 3π/2 e volta ao zero em 2π.

7. Conclusões

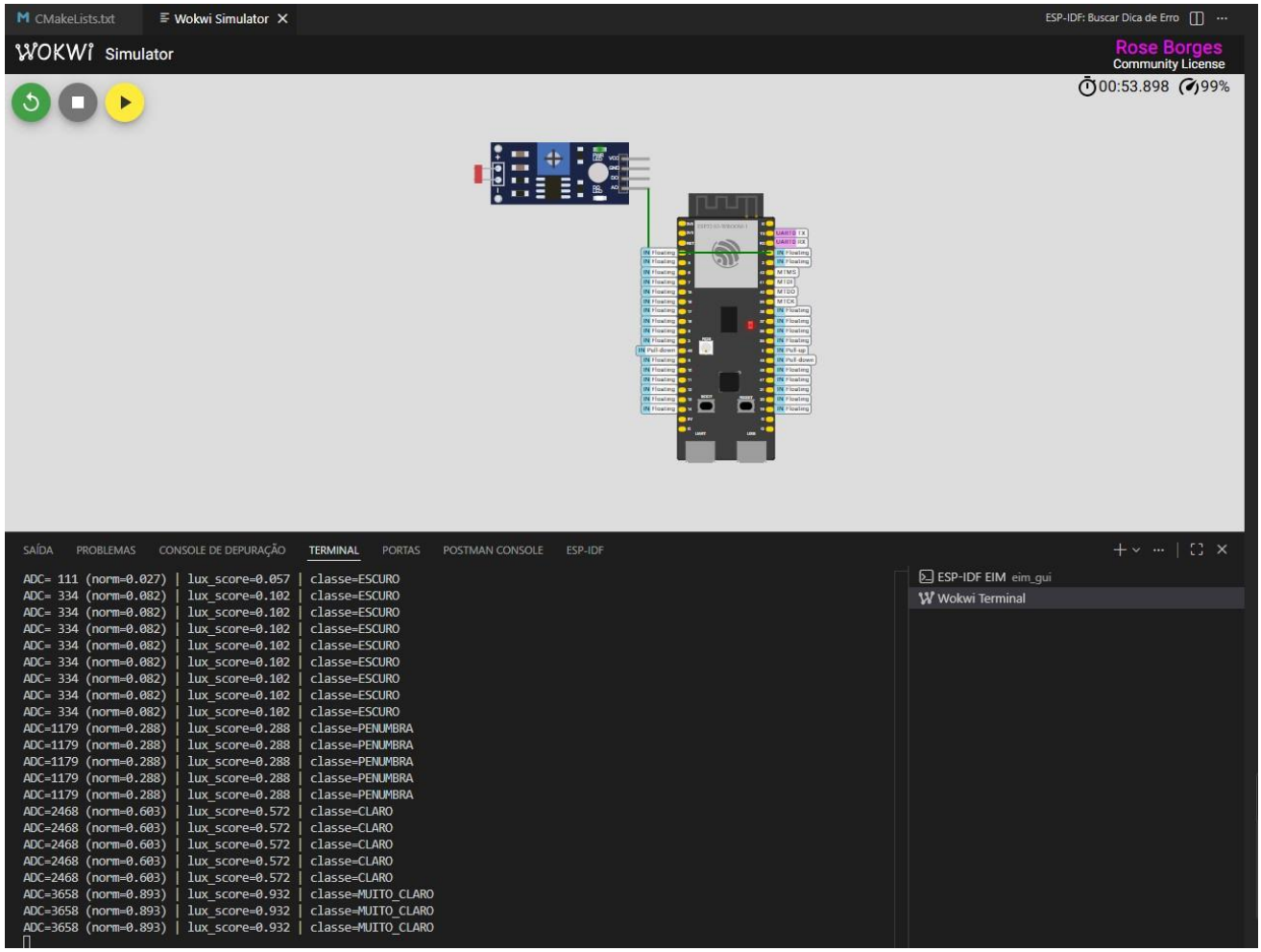
1. O fluxo TFLite Micro reduz um modelo Keras de 3 camadas a um *flatbuffer* de 3,3 KB que cabe em qualquer microcontrolador moderno.
2. A quantização int8 pós-treinamento é praticamente *free*: reduz o modelo pela metade com um custo de precisão pequeno (MAE 0,028 vs 0,017). Para o ESP32-S3 ainda permite acelerações específicas via ESP-NN.
3. A arquitetura do firmware deixa claros os pontos de extensão para uma nova aplicação: trocar `g_model[]`, ajustar `MicroMutableOpResolver` para os novos ops, mudar o `loop()` para ler um sensor real ao invés de gerar x artificial, e reaproveitar todo o resto.
4. A escolha entre kernels de referência (portáveis) e otimizados (ESP-NN/CMSIS-NN) é uma flag de build — útil saber em que contexto cada um se aplica (Wokwi exige referência; produção ganha 2-5× com os otimizados).
5. Limitações práticas observadas: TFLM aceita só um subconjunto de operadores TF; não há treino on-device; toda memória precisa ser estática. Esses limites guiam o projeto desde o treinamento — não adianta treinar um modelo cheio de ops exóticos que não vão ter kernel no microcontrolador.

8. Extra — Light Classifier

Implementei a aplicação extra opcional (+1 ponto): classificador de luminosidade em 4 zonas (ESCURO/PENUMBRA/CLARO/MUITO_CLARO) usando um LDR como sensor analógico no ADC1_CH0 (GPIO1) do ESP32-S3.

- **Sensor**: módulo `wokwi-photoresistor-sensor` ligado direto na ADC (sem pulldown externo — o módulo já tem condicionamento).
- **Dataset sintético**: `adc_norm` uniforme em [0,1] mapeado para `lux = sqrt(adc_norm)` com ruído gaussiano. Modelo aprende a curva côncava — não-trivial mas suave.
- **Mesma arquitetura** do Hello World (Dense(16)→Dense(16)→Dense(1)) e mesmo único operador (`FullyConnected`), então o `MicroMutableOpResolver<1>` e a tensor arena de 2 KB seguem servindo. Só muda o `model.cc`, o `loop()` (que agora lê ADC) e o `output_handler` (classificação por limiares).
- **Resultados no Wokwi**:

ADC raw	norm	lux_score	classe
111	0.027	0.057	ESCURO
1179	0.288	0.288	PENUMBRA
2468	0.603	0.572	CLARO
3658	0.893	0.932	MUITO_CLARO



Pipeline de retreino reaproveitou todo o ferramental do Hello World única integração nova foi o periférico `esp_adc/adc_oneshot` (5 linhas de setup + 1 leitura por ciclo).